ISTA 130 Lists and File I/O Worksheet

Write a function called `time_to_seconds` has one string parameter representing a time in format `"hours:minutes:seconds"` (e.g. `"10:3:7"`), and returns the equivalent number of seconds. There is more than one way to do this, but the `find` method may be helpful.

Write a function called `swap_case` that takes a string and returns a new string with all lowercase characters converted to upper and all upper to lower.

Write a function called `line_nums` that takes a filename (string) argument. It opens the file, reads it and writes a new file with the same name but the string `'_ln'` added to the filename stem. For example, if the filename is `'textfile.txt'`, the stem is `'textfile'` and new file for writing should be called `'textfile_ln.txt'`. Assume the filename has an extension and only one dot. The new file should contain the same text as the original but each line should start with its line number followed by a colon and a space. For example, if the tenth line in the original file is `'I heart Rocket.'`, the tenth line in the new file should be `'10: I heart Rocket.'`

Write a function called `word_search` that takes two string arguments, a filename and a word. The function prints "***** <word> *****" and then every line that contains the word with its line number added as above. (Hint: there are methods called `isupper`, `islower`, `upper`, and `lower`).

Write code that does the following. Create a list that has the values `"Arizona"`, `"California"`, and `"New Mexico"` in it. Add the value `"Utah"` to the end of the list. Add the value `"Oregon"` at the beginning of the list. Use a `for` loop to change all of the values in the list to uppercase. Use a `while` loop to change them all to lowercase. Eliminate the first element in the list. Eliminate the last. (Hint: `append`, `insert`, `pop`).

Write a function called `add_elements`. It takes two list arguments and returns a new list. Each element in the new list is the sum of the elements at the same position in the arguments. You may assume the lists are the same length. For example:

```
add_elements([1, 2, 1, 3], [7, 14, 6, 33])  -->  [8, 16, 7, 36].
```

Write a function called `join` that takes a list of strings and a string separator and returns a string consisting of the strings in the list separated by the separator. Example calls:

```
join(['I', 'heart', 'Rocket!'], ' ') --> 'I heart Rocket!'
join(['I', 'heart', 'Rocket!'], '--') --> 'I--heart--Rocket!'
```